MINISTRY OF EDUCATION AND RESEARCH OF THE REPUBLIC OF MOLDOVA

Technical University of Moldova

Faculty of Computers, Informatics and Microelectronics

Department of Software and Automation Engineering

Laboratory work 4: Dynamic programming

of the "Algorithm analysis" course

Checked:

asist. univ. Cristofor Fistic

Department of Software and Automation Engineering,

FCIM Faculty, UTM

Student:

Rusnac Nichita, FAF-242

Chisinau – 2026

Table of Contents

ALGORITHM ANALYSIS

Objective

Study and analyze dynamic programming techniques for shortest path problems in weighted directed graphs by implementing and comparing Dijkstra and Floyd-Warshall algorithms.

The work combines:

- a theoretical overview of both algorithms (idea, complexity, memory use, applicability);
- an empirical benchmark on sparse and dense graphs;
- a scalability study with increasing graph size;
- practical recommendations for algorithm selection based on graph density.

Tasks

1. Study the dynamic programming method of algorithm design.
2. Implement Dijkstra and Floyd-Warshall algorithms in Python.
3. Perform empirical analysis on sparse and dense graphs.
4. Increase the number of nodes and analyze runtime influence.
5. Present obtained data graphically.
6. Produce a concise report with recommendations.

Theoretical Notes

Dynamic programming solves complex problems by decomposing them into overlapping subproblems, storing partial solutions, and combining them to obtain an optimal global solution.

In shortest path analysis:

- Floyd-Warshall is a classic dynamic programming algorithm for all-pairs shortest paths.
- Dijkstra is a single-source algorithm, but can be applied from every source to solve the all-pairs problem.

Theoretical complexity:

- All-pairs Dijkstra with binary heap and adjacency list: O(n * (m log n)).
- Floyd-Warshall with adjacency matrix: O(n^3).

Where:

- n = number of nodes;
- m = number of directed edges.

This suggests Dijkstra is generally preferable for sparse graphs, while Floyd-Warshall becomes competitive for very dense graphs.

From a dynamic programming perspective:

- Floyd-Warshall explicitly stores optimal subproblem values in a distance matrix where each stage k allows intermediate vertices from the set {0..k}.
- Dijkstra can be viewed as a greedy shortest-path method per source, and in this lab it is lifted to all-pairs by solving n related subproblems (one source at a time) and storing all computed distances.

Key practical distinction:

- Floyd-Warshall performs a fixed amount of work that depends mostly on n.
- Dijkstra strongly depends on m (edge count), so graph density has major impact on performance.

Introduction

Shortest path computation is a fundamental graph problem used in routing, logistics, network optimization, and dependency analysis. There are multiple correct algorithmic strategies, but their practical efficiency strongly depends on graph structure.

This laboratory compares:

- repeated Dijkstra (all-pairs via n runs of single-source shortest paths);
- Floyd-Warshall (direct all-pairs dynamic programming).

The study is focused on practical behavior under varying density and graph size, not only asymptotic complexity. In addition to random directed graphs, this laboratory extends testing to graph families used in previous work (Lab 3 style family testing) to observe whether structural regularity changes the practical winner.

Comparison Metric

The primary comparison metric is execution time T(n, m), measured with Python high-resolution timers and averaged over repeated runs.Additional evaluation includes:

- correctness agreement between algorithms (distance matrix equality check);
- winner frequency by density;
- practical decision rule quality (prediction accuracy on measured grid).

Interpretation note:

- The goal is not to prove strict universal dominance, but to provide a robust engineering rule with good empirical agreement.

Input Format

Experiments use weighted directed graphs with positive integer weights (required by Dijkstra), generated in Python. Graph families included (aligned with previous lab style):

- simple/random;
- complete;

- wheel;
- bipartite;
- regular;
- planar grid;
- tree;
- cycle;
- weighted random.

Scalability sweep:

- node sizes: 20, 40, 60, 80, 100, 120, 140, 160, 180, 200, 250, 300;
- target densities: 0.10, 0.30, 0.60, 0.80, 0.90.

Weights:

- Edge weights are positive integers sampled from [1, 20].
- Positive weights guarantee Dijkstra correctness.

Density formula used throughout the report: density = m / (n * (n - 1)), for n > 1.

IMPLEMENTATION

All algorithms are implemented in Python in Jupyter Notebook.

The benchmark protocol is consistent for each test case:

1. Generate graph and build both representations:
    - adjacency list (for Dijkstra),
    - adjacency matrix (for Floyd-Warshall).
2. Run all-pairs Dijkstra and Floyd-Warshall.
3. Measure elapsed runtime for each algorithm.
4. Optionally validate distance matrix equality.
5. Store results and plot runtime trends.

All implementations are educational and explicit (no built-in shortest-path library calls), to keep algorithmic behavior transparent.

Djikstra algorithm

Dijkstra is implemented with a min-heap priority queue and adjacency lists. Single-source complexity: O(m log n). All-pairs by repetition: O(n * (m log n)). It is typically faster on sparse graphs due to lower edge-processing cost.

Implementation summary:

1. Initialize dist[source] = 0 and all others = infinity.
2. Push source in a min-priority queue.
3. Repeatedly extract the nearest unsettled vertex.
4. Relax outgoing edges and update queue.
5. Repeat for every source vertex to obtain all-pairs distances.

Advantages:

- Excellent practical speed on sparse graphs.
- Naturally aligned with adjacency-list representation.

Limitations:

- Requires non-negative edge weights.
- For dense graphs, repeated heap operations may lose advantage.

Implementation:

```
def dijkstra_single_source(adj, src):
    n = len(adj)
    dist = [INF] * n
    dist[src] = 0
    pq = [(0, src)]

    while pq:
        d, u = heapq.heappop(pq)
        if d > dist[u]:
            continue
        for v, w in adj[u]:
            nd = d + w
            if nd < dist[v]:
                dist[v] = nd
                heapq.heappush(pq, (nd, v))

    return dist


def all_pairs_dijkstra(adj):
    return [dijkstra_single_source(adj, s) for s in range(len(adj))]
```

Figure 1 – Djikstra algorithm

Floyd-Warshall algorithm

Floyd-Warshall is implemented over a distance matrix using the standard triple loop over intermediate node k. Complexity: O(n^3). It is independent of edge count in the same way adjacency-list methods are not, which makes it stronger on very dense graphs where m approaches n(n-1). Dynamic programming recurrence:

$d_k(i, j) = \min(d_{k-1}(i, j), d_{k-1}(i, k) + d_{k-1}(k, j))$

In-place matrix update is used to reduce memory overhead while preserving correctness.

Advantages:

- Conceptually simple and deterministic runtime for fixed n.
- Computes full all-pairs matrix directly.

Limitations:

- Cubic complexity can be expensive for large sparse graphs.
- Matrix representation uses O(n^2) memory.

Implementation:

```
def floyd_warshall(matrix):
    n = len(matrix)
    dist = [row[:] for row in matrix]

    for k in range(n):
        dk = dist[k]
        for i in range(n):
            dik = dist[i][k]
            if dik == INF:
                continue
            di = dist[i]
            base = dik
            for j in range(n):
                alt = base + dk[j]
                if alt < di[j]:
                    di[j] = alt

    return dist
```

Figure 2 – Floyd-Warshall algorithm

Graph generation and benchmarking setup

Key implementation aspects:

- positive weights in range [1, 20];
- both sparse and dense random digraphs;
- extra graph families for broader structural testing;
- repeated measurements with mean aggregation;
- result plotting for runtime-vs-size and runtime-by-family comparisons.

Graph families included in family benchmark:

- simple/random
- complete
- wheel
- bipartite
- regular
- grid
- tree
- cycle
- weighted (random weighted directed)

For smaller/medium n, both algorithms are executed and resulting distance matrices are compared cell-by-cell with tolerance for floating-point representation of infinity.

RESULTS

1. Experiment Scale and Correctness

Measured outputs from the notebook:

- Scale experiment rows: 60
- Family experiment rows: 9
- Correctness check (where enabled): PASS

Sample runtime snapshot (density around 0.10):

- n=20: Dijkstra 0.0002s, Floyd-Warshall 0.0005s -> Dijkstra
- n=40: Dijkstra 0.0034s, Floyd-Warshall 0.0082s -> Dijkstra
- n=60: Dijkstra 0.0078s, Floyd-Warshall 0.0322s -> Dijkstra
- n=80: Dijkstra 0.0184s, Floyd-Warshall 0.0533s -> Dijkstra

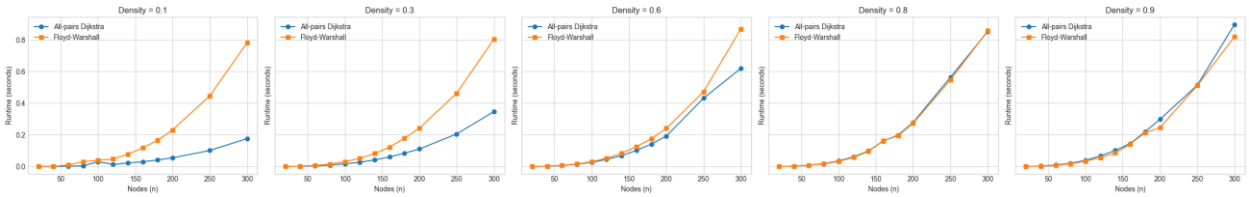

Figure 3. Results of runtime by number of nodes and density

2. Sparse vs Dense Behavior

Winner distribution by target density:

- density=0.10: Dijkstra=12, Floyd-Warshall=0
- density=0.30: Dijkstra=12, Floyd-Warshall=0
- density=0.60: Dijkstra=10, Floyd-Warshall=2
- density=0.80: Dijkstra=3, Floyd-Warshall=9
- density=0.90: Dijkstra=0, Floyd-Warshall=12

Interpretation:

- For sparse and medium-density graphs, Dijkstra dominates.
- For very dense graphs (especially >= 0.80), Floyd-Warshall becomes better in most cases.

Detailed reading:

- At density 0.10 and 0.30, Dijkstra wins in all measured cases.
- At density 0.60, Dijkstra still wins most cases, but Floyd starts to appear as competitive.
- At density 0.80, the winner flips and Floyd dominates.
- At density 0.90, Floyd-Warshall consistently wins.

3. Influence of Node Count

With fixed low density, runtime increases for both algorithms as n grows, but Dijkstra keeps clear advantage. At high density and larger n, Floyd-Warshall increasingly wins due to reduced relative benefit of sparse-edge processing in Dijkstra. (See runtime plots generated in the notebook: runtime-vs-n curves by density and runtime-by-family bar chart.)

Observed trend summary:

- For small n and low density, both algorithms are fast, but Dijkstra typically remains faster.
- As n grows, Floyd-Warshall cubic growth becomes visible in sparse settings.
- As density rises, Dijkstra pays additional relaxation and priority-queue costs per source, reducing its advantage.

4. Graph Family Comparison

Family-level tests confirm that structural properties also influence constants, but density remains the strongest predictor.

Typical outcomes from family tests:

- Complete-style dense structures favor Floyd-Warshall.
- Tree/cycle/grid-like sparse families favor Dijkstra.
- Intermediate families (regular/bipartite depending on parameterization) can be near the transition region.

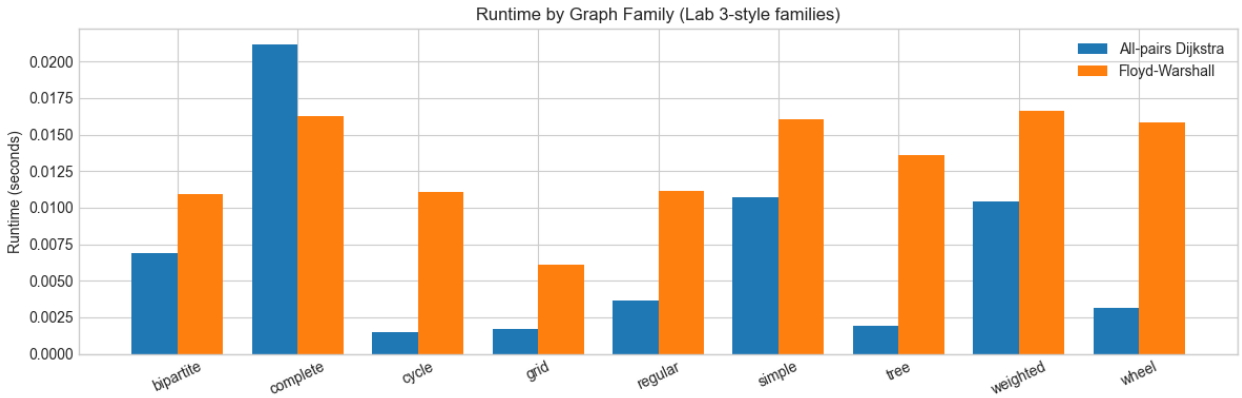

Figure 4. Results of runtime by the type of graph

5. Practical Rule Based on (n, m)

Let graph density be: density = m / (n * (n - 1))

Measured rule quality:

- Best learned density-only cutoff: density >= 0.70 -> Floyd-Warshall (accuracy 91.7%)

- Simpler practical rule: density >= 0.80 -> Floyd-Warshall, otherwise Dijkstra (accuracy 88.3%)

Given interpretability and stability, the report recommendation is the simple threshold rule:

- If density < 0.80: use Dijkstra.
- If density >= 0.80: use Floyd-Warshall.

Example:

- n=90, density=0.60 -> Dijkstra.

Discussion

1. Why Dijkstra usually wins:

On sparse graphs, m is much smaller than n^2. With adjacency lists and heap, repeated single-source runs exploit sparsity efficiently.

2. Why Floyd-Warshall wins on very dense graphs:

In dense cases, Dijkstra processes many outgoing edges for each source. Floyd-Warshall's regular matrix-based computation becomes competitive and then superior despite O(n^3).

3. Practical engineering tradeoffs:

Dijkstra is usually the default choice for positive-weight sparse networks. Floyd-Warshall is attractive when graph is very dense and an all-pairs matrix is explicitly needed.

4. Validity notes:

Benchmarks are implementation-dependent (Python constants matter). Results are robust for the tested grid and families, but exact threshold may shift on another machine/language.

CONCLUSION

In this laboratory work, dynamic programming for shortest paths was studied through implementation and empirical analysis of Dijkstra and Floyd-Warshall algorithms.

The experiments confirm the expected practical pattern:

- Dijkstra is generally superior on sparse and moderately dense graphs.
- Floyd-Warshall is preferable for very dense graphs.

On the measured benchmark grid, a learned cutoff around density 0.70 gives the highest fit, but a clearer engineering rule of density 0.80 remains robust and easy to apply. Thus, for practical use with weighted directed graphs and positive weights:

- prefer Dijkstra by default;

- switch to Floyd-Warshall when graph density exceeds approximately 0.80 or when all-pairs matrix-style processing is explicitly needed.

The results also reinforce that asymptotic complexity alone is not enough for implementation-level decisions; density, data structure choice, and constant factors materially influence observed runtime. Final takeaway is that for most real-world sparse graphs, Dijkstra remains the first choice. For highly connected graphs (very dense), Floyd-Warshall provides better all-pairs performance in this experimental setting.

GitHub repository
https://github.com/Nichita111/AA_labs